



Introdução: OWASP e a Importância do Rate Limiting

OWASP é uma organização dedicada a melhorar a segurança de software. "Lack of Resources Rate Limiting" significa não limitar o uso dos recursos da aplicação. Esta falha pode causar ataques como DoS, que vêm crescendo 37% ao ano.



O Problema Detalhado: Abuso de Recursos

Ataques de Força Bruta

Tentativas ilimitadas de senhas ameaçam contas de usuários.

Scraping Excessivo

Coleta massiva de dados que sobrecarrega servidores.

Exploração de Funcionalidades

Uso indiscriminado de recursos como login e busca.

Impacto Financeiro

Cada ataque DoS custa cerca de \$30.000.

Consequências da Falta de Rate Limiting

- Denial of Service
(DoS) sistema indisponível para usuários legítimos.
- Degradação da Performance
Lentidão prejudica experiência do usuário.
- Impacto Financeiro
Perda de receita e custos altos para recuperação.
- Exemplo Real
E-commerce perdeu \$500.000 em ataque de scraping.



Tipos Comuns de Ataques pela Falta de Rate Limiting

Força Bruta em Login

Tentativas massivas de acesso via formulário.

Abuso de APIs Públicas

Sem autenticação e limitação, APIs ficam vulneráveis.

Spam em Formulários

Envio massivo de mensagens irritantes e perigosas.

DDoS Multicamadas

82% dos ataques incluem a camada da aplicação.

How To Security Protected

Your layer Fronzade from the nest of protected ints
outouriny layer the accamy in data.



Estratégias de Rate Limiting: Abordagem em Camadas



Limitação por IP
Controla requisições
pelo endereço IP.



Limitação por
Usuário
Monitora uso para cada
usuário autenticado.



Limitação por
Endpoint
Regula acessos a
endpoints específicos.



Algoritmos
Token Bucket para
simplicidade; Leaky
Bucket para suavizar
picos.



Implementação Prática de Rate Limiting

- 1 Bibliotecas e Middleware**
Exemplos: Flask-Limiter para Python, ferramentas Node.js.
- 2 Configuração de Servidores**
Nginx e Apache com módulos de rate limiting.
- 3 Integração com CDNs**
Cloudflare e Akamai oferecem proteção e limitação.
- 4 Código Exemplo**
`Limiter(app, key_func=get_remote_address, default_limits=["200 per day, 50 per hour"])`

Monitoramento e Ajuste Fino do Rate Limiting

Monitoramento de Métricas

Acompanhe requisições, falhas e desempenho.

Análise de Logs

Identifique padrões e tentativas de ataque.

Ajuste Dinâmico

Regras flexíveis para adaptação a novas ameaças.

Ferramentas

Prometheus e Grafana para monitoramento em tempo real.

Conclusão: Protegendo Aplicações com Rate Limiting

Defesa Essencial

Rate limiting previne abusos e ataques.

Benefícios

Garante disponibilidade, performance e segurança.

Próximos Passos

Auditoria, implementação e manutenção contínua.

Recomendação OWASP

Rate limiting como padrão de segurança web.



Próximos Slides: Exemplos Práticos

Nesta seção, apresentaremos casos reais que ilustram o rate limiting em ação.

Veremos vulnerabilidades comuns e como implementar proteções eficientes.

1 Exemplo 1

Vulnerabilidade sem rate limiting detectada.

2 Exemplo 2

Implementação prática de rate limiting para proteção.

Exemplo Prático 1 – Vulnerabilidade Sem Rate Limiting

O código Node.js a seguir apresenta um servidor Express sem limitação de taxa.

Sem controle, o endpoint /login permite ataques de força bruta.

Qualquer pessoa pode tentar múltiplas combinações de usuário e senha sem bloqueio.

Essa falha expõe o sistema a invasões e perda de dados.

```
const express = require('express');
const app = express();

app.get('/login', (req, res) => {
  const { username, password } = req.query;
  if (username === 'admin' && password === '1234') {
    res.send('Login bem-sucedido');
  } else {
    res.send('Credenciais inválidas');
  }
});

app.listen(3000, () => {
  console.log('Servidor rodando na porta 3000');
});
```

Exemplo Prático 2 – Implementando Rate Limiting

Agora o sistema limita a 5 tentativas (`max: 5`), de login a cada 15 minutos por IP.

Ao exceder o limite, o usuário recebe uma mensagem para aguardar antes de tentar novamente.

```
const express = require('express');
const rateLimit = require('express-rate-limit');
const app = express();

// Cria um limitador de taxa
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 5, // Limita a 5 requisições
  message: 'Muitas tentativas de login. Tente novamente em  
15 minutos.',
});

// Aplica o rate limiter apenas para a rota de login
app.use('/login', loginLimiter);
```

```
app.get('/login', (req, res) => {
  const { username, password } = req.query;
  if (username === 'admin' && password === '1234') {
    res.send('Login bem-sucedido');
  } else {
    res.send('Credenciais inválidas');
  }
});

app.listen(3000, () => {
  console.log('Servidor rodando na porta 3000');
});
```

Explicação do Código de Rate Limiting

express-rate-limit: Biblioteca essencial para controlar a taxa de requisições por IP.

windowMs: Define o intervalo de tempo (ex: 15 minutos) para aplicar o limite.

max: Especifica o número máximo de tentativas permitidas nesse período (ex: 5).

message: Texto exibido quando o limite é ultrapassado, orientando o usuário.

Outros Exemplos de Rate Limiting

Rate Limiting em Python com Flask

A biblioteca Flask-Limiter controla requisições usando o IP do cliente.

- Limite configurado para 5 requisições a cada 15 minutos
- Proteção simples para rotas sensíveis, como login
- Respostas HTTP indicam tentativas inválidas

Rate Limiting no Nginx (Servidor)

Configuração direta no servidor proxy para controle rápido.

- Limita 5 requisições por minuto por IP para a rota /login
- Bursts permitidos para lidar com picos controlados
- Melhora eficiência antes mesmo do app ser atingido

Rate Limiting em Python com Flask

```
from flask import Flask, request, jsonify
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

app = Flask(__name__)
limiter = Limiter(app, key_func=get_remote_address)
```

```
@app.route("/login")
@limiter.limit("5 per 15 minute") # Limita a 5 requisições
por 15 minutos
def login():
    username = request.args.get('username')
    password = request.args.get('password')
    if username == "admin" and password == "1234":
        return "Login bem-sucedido"
    return "Credenciais inválidas", 400

if __name__ == "__main__":
    app.run(debug=True)
```

Exemplo em Nginx (Server Side Rate Limiting)

```
http {  
    limit_req_zone $binary_remote_addr zone=mylimit:10m rate=5r/m; # Limita a 5 requisições  
    por minuto  
    server {  
        location /login {  
            limit_req zone=mylimit burst=5 nodelay;  
            proxy_pass http://localhost:3000;  
        }  
    }  
}
```